



SEN381

GERALD ENRIGHT 577830 | KELETSO MAROTA 601632 | MOGAU MALOPE 600192

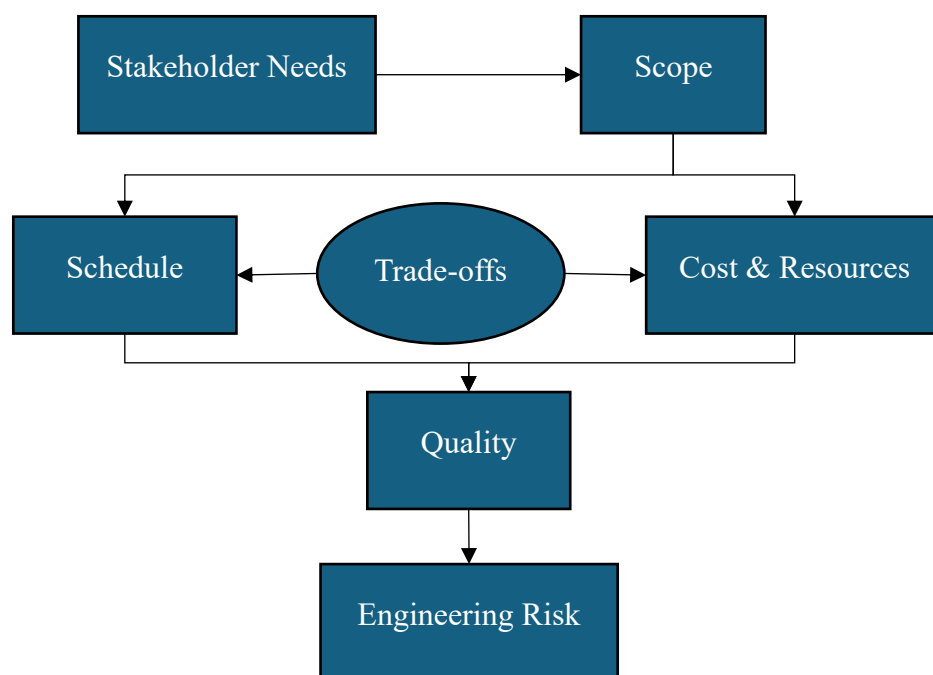
Table of Contents

Question 1 - Software Engineering Beyond Programming	3
Question 2 - Thinking Ahead: Decisions Across the Software Lifecycle.....	4
1. Six Critical Software Lifecycle Decisions	4
2. Analysis of the Six Decisions	4
System & Security Architecture.....	4
Data Architecture & Persistence Strategy	5
Non-Functional Requirements & Infrastructure Alignment	5
DevOps Automation & CI/CD Strategy	6
Integration Strategy & External API Boundaries.....	6
Regulatory Compliance, Governance & Data Privacy	7
3. Decision Horizon Map	7
4. Why Waiting Until Implementation Can Be Too Late.....	8
Question 3 - Evidence-Based Technology Stack Decisions	9
Technology stack options:.....	9
Lifecycle comparison.....	9
Weight justification	11
Weighted decision matrix.....	11
Interpretation and limitations.....	12
Critical conclusion	12
Question 4 - Responsible AI in Software Engineering	12
Uses Of Generative AI Across Engineering Activities	13
Risks and limitations.....	13
Human Verification and Control Process.....	13
Accountability Question	14
References.....	15

AI Research & Verification Record	16
---	----

Question 1 - Software Engineering Beyond Programming

Successful software development requires far more than selecting a programming language or writing functional code because software engineering manages essential complexity, like the inherent challenge of defining and modelling real world domain problems, unlike syntax that only addresses accidental complexity (Alzayed, 2024). Functional code merely confirms that an execution can parse instructions with no guarantee that the system is scalable, maintainable, or economically viable over time.



The above model demonstrates how software engineering operates as an interconnected web of computing constraints. Stakeholders establish scope, which decides the base schedule and required costs/resources. Engineers are often forced to trade quality by taking shortcuts when time and budget is fixed, accumulating technical debt. Technical debt escalates engineering risk which increases the likelihood of outages, vulnerabilities and future reworks.

A real-world illustration of this dynamic is documented in modern empirical case studies on public health and government platform deployments, such as the Healthcare.gov architectural analysis (Ebad, 2020). Facing a legally mandated launch date, project managers required all users to establish an account and complete identity verification before browsing insurance plans. To meet the deadline without altering the scope, engineers bypassed full end-to-end load testing and built tight, synchronous dependencies across external databases. On launch day massive traffic choked the mandatory authentication gateway, causing cascading system-

wide timeouts and crashing for over 99% of visitors. The resulting technical debt required a multi-million-dollar emergency overhaul post launch.

Question 2 - Thinking Ahead: Decisions Across the Software Lifecycle

1. Six Critical Software Lifecycle Decisions

Decision / concern	First considered	Later implementation / effect	Main risk if delayed
System & Security Architecture	Concept/inception and early requirements/design, before major structural commitments are made.	During detailed design and development, then continuously during testing, deployment and operations.	Security and architectural changes may require restructuring already-built components, increasing rework and leaving vulnerabilities or architectural debt.
Data Architecture & Persistence Strategy	Requirements analysis and domain/data modelling, before application features become dependent on a persistent structure.	During schema/database design and development, then through migration, scaling, backup and recovery operations.	An unsuitable persistence model can become embedded throughout the codebase, making migrations, performance improvements and consistency changes more expensive.
Non-Functional Requirements & Infrastructure Alignment	Early discovery and requirements elaboration, before the system's design is fixed around functional requirements alone.	During architecture/design, integration and performance testing, and especially during deployment and production operations.	A system may satisfy functional requirements but fail under realistic load or operational conditions, forcing architecture changes late in the lifecycle.
DevOps Automation & CI/CD Strategy	Project setup/Sprint 0, once the development workflow and environments are established.	Throughout development and testing, and continuously during release and operations.	Manual integration and release processes accumulate friction and make defects harder to isolate.
Integration Strategy & External API Boundaries	High-level architecture and design, before independent components or third-party integrations are deeply coupled.	During API/component development, integration testing and production operations.	Components can become tightly coupled, and changes to an external service can propagate failures or require widespread rework.
Regulatory Compliance, Governance & Data Privacy	Early requirements analysis and system specification, before data structures and workflows make compliance assumptions difficult to change.	During design and development and especially at deployment, audit and ongoing operations.	Compliance controls may need to be retrofitted into core workflows and data stores, increasing cost and potentially creating legal and operational exposure.

2. Analysis of the Six Decisions

System & Security Architecture

What: Establish the system's major components, responsibilities, trust boundaries, security controls and architectural relationships.

First considered: Concept/inception and early requirements/design, before major structural commitments are made.

Implemented/operationally significant: During detailed design and development, then continuously during testing, deployment and operations.

Information needed: Business and stakeholder goals; security requirements; threat/risk information; expected scale; trust boundaries; integration constraints; quality attributes.

Later activities affected: Component decomposition, API boundaries, authentication/authorisation, deployment topology, testing and change management.

If postponed: Security and architectural changes may require restructuring already-built components, increasing rework and leaving vulnerabilities or architectural debt. ISO/IEC/IEEE 42010:2022 provides a current framework for expressing architecture descriptions, while NIST SP 800-53 provides a broad catalogue of security and privacy controls (ISO, 2022; NIST, 2020/2025).

Evidence: ISO/IEC/IEEE 42010:2022; NIST SP 800-53 Rev. 5.

Data Architecture & Persistence Strategy

What: Decide how information will be modelled, stored, accessed, protected, backed up and evolved.

First considered: Requirements analysis and domain/data modelling, before application features become dependent on a persistent structure.

Implemented/operationally significant: During schema/database design and development, then through migration, scaling, backup and recovery operations.

Information needed: Domain relationships; data volume and growth; read/write patterns; consistency and transaction needs; retention; recovery objectives; security/privacy requirements.

Later activities affected: Data-access code, queries, indexes, caching, reporting, migrations, integration and disaster recovery.

If postponed: An unsuitable persistence model can become embedded throughout the codebase, making migrations, performance improvements and consistency changes more expensive. Kleppmann emphasises reliability, scalability, maintainability and evolvability as central concerns in data-intensive systems (Kleppmann, 2017).

Evidence: Kleppmann (2017).

Non-Functional Requirements & Infrastructure Alignment

What: Define measurable quality and operational targets such as performance, availability, capacity, security and maintainability, and align them with infrastructure assumptions.

First considered: Early discovery and requirements elaboration, before the system's design is fixed around functional requirements alone.

Implemented/operationally significant: During architecture/design, integration and performance testing, and especially during deployment and production operations.

Information needed: Expected users and workloads; peak concurrency; latency targets; availability objectives; recovery expectations; security requirements; operating constraints.

Later activities affected: Architecture, server/cloud topology, caching, load balancing, resilience, testing and acceptance criteria.

If postponed: A system may satisfy functional requirements but fail under realistic load or operational conditions, forcing architecture changes late in the lifecycle. ISO/IEC 25010:2023 provides a product quality model intended to support requirements, design, testing, quality assurance and evaluation throughout the lifecycle (ISO, 2023).

Evidence: ISO/IEC 25010:2023; Wiegers and Beatty (2013).

DevOps Automation & CI/CD Strategy

What: Establish source-control, build, automated testing, integration and deployment practices that allow changes to move through a controlled delivery pipeline.

First considered: Project setup/Sprint 0, once the development workflow and environments are established.

Implemented/operationally significant: Throughout development and testing, and continuously during release and operations.

Information needed: Repository strategy; environments; deployment targets; release frequency; test automation; secrets/configuration handling; rollback requirements.

Later activities affected: Integration, regression testing, release management, deployment reliability and feedback cycles.

If postponed: Manual integration and release processes accumulate friction and make defects harder to isolate. Humble and Farley describe automated build, test and deployment practices as foundations for reliable, low-risk delivery (Humble and Farley, 2010).

Evidence: Humble and Farley (2010).

Integration Strategy & External API Boundaries

What: Define how internal components and external services communicate, including interface contracts, protocols, error handling, versioning and dependency boundaries.

First considered: High-level architecture and design, before independent components or third-party integrations are deeply coupled.

Implemented/operationally significant: During API/component development, integration testing and production operations.

Information needed: Interface contracts; protocol choices; data formats; dependency reliability; rate limits; authentication; versioning; failure and timeout behaviour.

Later activities affected: Component integration, testing, monitoring, error recovery, API versioning and change management.

If postponed: Components can become tightly coupled, and changes to an external service can propagate failures or require widespread rework. OWASP's API Security Top 10 highlights risks including broken authorisation, security misconfiguration, improper API inventory and unsafe consumption of APIs, reinforcing the need to consider API boundaries and security early (OWASP, 2023).

Evidence: Newman (2021); OWASP (2023).

Regulatory Compliance, Governance & Data Privacy

What: Determine legal, regulatory and governance requirements for information handling, access, retention, auditability, consent and security.

First considered: Early requirements analysis and system specification, before data structures and workflows make compliance assumptions difficult to change.

Implemented/operationally significant: During design and development and especially at deployment, audit and ongoing operations.

Information needed: Applicable laws and policies; categories of personal data; processing purposes; retention periods; access rules; security obligations; data-location requirements; audit needs.

Later activities affected: Data models, consent and access workflows, encryption, logging, retention/deletion mechanisms, testing and operational governance.

If postponed: Compliance controls may need to be retrofitted into core workflows and data stores, increasing cost and potentially creating legal and operational exposure. The European Commission explains that privacy by design requires safeguards to be incorporated at the earliest stages of processing design (European Commission, n.d.). NIST SP 800-53 also integrates security and privacy controls within risk management.

Evidence: European Commission (n.d.); NIST SP 800-53 Rev. 5.

3. Decision Horizon Map

The map distinguishes when each concern should first be considered from when it becomes more operationally visible or is implemented. The arrows show that early decisions create constraints and inputs for later design, development, testing, deployment and operations.

Concern	Inception / Discovery	Requirements / Domain	Architecture / Design	Development / Integration	Testing / Deployment	Operations / Change
System & Security Architecture	●	→	●	→	●	→
Data Architecture & Persistence	○	●	●	→	●	→
NFRs & Infrastructure	●	●	→	→	●	●
DevOps & CI/CD	○	○	●	●	→	●
Integration & API Boundaries	○	○	●	●	→	●
Compliance & Data Privacy	●	●	●	→	●	●

Legend: ● = early consideration / decision point; ○ = lower emphasis at that stage; → = downstream dependency or continuing effect.

4. Why Waiting Until Implementation Can Be Too Late

Waiting until implementation can be too late because implementation occurs after important constraints have already been established. By that point, data models may be embedded in application code, interfaces may have been adopted by multiple components, deployment assumptions may be built into the delivery process, and security or privacy requirements may affect existing workflows. A late change therefore has a wider blast radius: it can require redesign, migration, regression testing, retraining, operational changes or replacement of already-built components. Early consideration does not mean that every decision is permanently fixed; rather, it means that the team identifies the decision, gathers the information needed to evaluate it, records assumptions and preserves opportunities to change direction before the cost of change becomes high.

The six concerns are also interdependent. Architecture affects data and integration boundaries; NFRs influence infrastructure and testing; compliance affects data and security design; and CI/CD affects how safely changes can be introduced. This means lifecycle thinking is not simply a chronological checklist. It is a way of identifying dependencies early enough for evidence-based decisions to remain feasible.

Question 3 - Evidence-Based Technology Stack Decisions

This comparison is between 2 differing approaches for building a modern web-based application.

Technology stack options:

- TypeScript stack (React, NestJS, Node.js, PostgreSQL): This uses react for user interface and frontend in general, with NestJS/Node.js for the backend. It allows for end-to-end TypeScript with a structured and modular buildout.
- Java stack (html, CSS, Java, JavaScript): Using Java API/Spring boot with base html, CSS and JavaScript, this approach provides a Java server ecosystem with production-orientated conventions and tooling.

PostgreSQL will be used for both approaches in terms of database buildout, because it offers the best developer and community support, making it easier to get solutions for specific problems and many tutorials for help implementing features.

Lifecycle comparison

Criteria	TypeScript stack	Java stack
Team capability and learning curve	One TypeScript language across client and server reduces context switching. NestJS supplies a modular structure, but its Node package ecosystem still requires some research.	Java tech stack requires competence in two languages and readability of code may prove to be difficult as the project grows. The initial learning curve is higher.
Security ecosystem	Guards, validation, authentication packages and Node security practices are readily available.	Spring Boot makes use of the broad Spring Security ecosystem and provides common security-related

		production features (Spring, 2026).
Maintainability	NestJS promotes modular, loosely coupled and testable applications. This makes later bug fixing easier (NestJS, 2026).	It supports long-lived systems, but excessive framework complexity can increase maintenance cost (Spring, 2026).
Testing and tooling support	Modular design support makes testing easier. JavaScript/TypeScript tooling is extensive (NestJS, 2026).	Mature Java build and Spring testing supports enterprise development. Costly configuration and slower feedback may occur in larger projects.
Deployment/platform compatibility	Runs wherever a supported Node.js runtime environment is available. A simpler build pipeline is better for many web applications.	Runs as a stand-alone JAR or a container. JVM availability is broad, but resource requirements can be costly (Spring, 2026).
Development and operational cost	Open-source components and community support reduce research overhead. Hosting cost depends on workload and service choices.	Open-source components avoid licence fees, but specialist Java skills and potentially higher compute use can greatly increase cost.
Future change implications	Modularity and shared types allow for fast feature change. Package upgrades must be actively and closely monitored.	Has strong conventions. Major framework/JDK upgrades and integration complexity still require extensive pre-planning.

Weight justification

A general public-facing information system is reflected in weights. Due to the potential for user harm and deployment constraints, security is given the highest weight (20%). Because they have a significant impact on delivery risk and overall lifecycle effort, team capability, maintainability, testing/tooling, and future change each receive 15%. Although both open-source stacks are widely deployable and their actual prices depend on workload, hosting, service levels, and the team, deployment compatibility and cost both receive 10%.

Weighted decision matrix

Scoring scale in terms of fit: 1 = poor, 2 = decent, 3 = strong fit.

% Fit will be calculated as follows: $((\text{sum}(\text{weight} * \text{score})) / \text{total possible}) * 100$

Typescript stack: A

Java stack: B

Criteria	Weight %	Score: A	Weight * score: A	Score: B	Weight * score: B
Team capability and learning curve	15	3	45	2	30
Security ecosystem	20	2	40	3	60
Maintainability	15	3	45	3	45
Testing and tooling support	15	3	45	3	45
Deployment/platform compatibility	10	3	30	2	20
Development and operational cost	10	3	30	2	20

Future change implications	15	3	45	3	30
TOTAL	100	20	280	14	250
% Fit:	100		93		83

Interpretation and limitations

Under these assumptions, Stack A scores 93% and Stack B 83%. The 10% difference is prevalent enough to justify a preference, but is not universal. Stack B gains from security, maintainability and testing conventions; Stack A gains from a lower learning barrier and likely lower initial team training overhead. A team already proficient in TypeScript, a short deadline or a low-memory hosting constraint could come to the same result. Conversely, strict governance, complex domain rules or strong Java capability could increase Stack B's advantage.

The matrix simplifies uncertainty: ordinal scores can create false precision, weights can change from stakeholder judgements, criteria can overlap, and no prototype or workload benchmark has been performed.

Critical conclusion

Due to the contextual nature of suitability, the most popular or highest-scoring stack is not always accurate. A popular stack might provide expertise and community support, but it might still clash with security requirements, the infrastructure that is already in place, the timeline, the budget, or the operational capacity. Similarly, a weighted matrix does not take the role of stakeholder validation, risk analysis, technical testing, or responsible engineering judgment; it merely makes assumptions evident. Therefore, the stack that best satisfies confirmed project needs and constraints, rather than the alternative with the best reputation or a little higher generic score, is the justifiable decision.

Question 4 - Responsible AI in Software Engineering

In present modern software engineering generative AI operates as a dynamic force multiplier rather than an autonomous engineer. When effectively integrated the human effort shifts from low-level syntax construction to high-level architectural validation and design (Tosi, 2024).

Uses Of Generative AI Across Engineering Activities

Requirements & Specifications Expansion	AI models analyse stakeholder notes to draft initial user stories, edge-case scenarios and acceptance criteria. Revealing vague or hidden expectations early.
Automated Test Scaffolding & Edge-Case Synthesis	Generative AI creates unit-, integration- and property-based test suites from functional specifications. Thus, speeding up the testing process.
Boilerplate Code & Infrastructure Generation	AI automates repetitive tasks; like API controllers, data transfer objects and deployment manifests; reducing operational overhead.

Risks and limitations

Security Vulnerability Injection	Models trained on public repositories often suggest poorly written code containing known vulnerabilities, such as SQL injection vectors or unsafe memory management (Negri-Ribalta, 2024).
Plausible Hallucinations & Automation Bias	Developers often face psychological priming toward trusting AI generated code. Because generated logic appears authoritative, engineers often overlook subtle semantic or boundary-condition bugs (Negri-Ribalta, 2024).
Silent Technical Debt & Deskilling	Uncritical adoption of AI output inflates codebases faster than human teams can comprehend. Over reliance degrades foundational debugging and architectural skills over time (Tosi, 2024).

Human Verification and Control Process

To ensure safety before material AI-generated output is committed to a codebase teams should enforce a Three-Gate process:

Gate 1: Static Analysis and Linting	Automated static application security testing tools scan the generated output for vulnerabilities and style violations.
Gate 2: Automated Test Execution	The code must pass existing regression suites alongside newly written, human verified unit tests.
Gate 3: Named Human Peer Review	A human engineer evaluates the output for logical correctness, maintainability and architectural compliance.

Accountability Question

The human engineer is accountable for the work that they accept. Before AI was introduced into the workplace, unreliable work would be developed by the interns or untrained personnel and while these people had to take responsibility for a while their work could not affect production without the approval of a trained professional who would be held responsible for the design and decisions that affect production. In the modern workplace AI is essentially another version of untrained personnel in which case we can hold it responsible for its design however the person how decides to apply that design to production must be held responsible.

References

- Alzayed, A., 2024. Evaluating the role of requirements engineering practices in the sustainability of electronic government solutions.. *Sustainability*, 16(1), p. 433.
- Ebad, S., 2020. Healthcare software design and implementation - A project failure case.. *Software: Practice and Experience*, 50(7), pp. 1258-1276.
- Negri-Ribalta, C. G.-S. R. S. A. a. L. G., 2024. A systematic literature review on the impact of AI models on the security of code generation.. *Frontiers in Big Data*, Volume 7, p. 1386720.
- NestJS, 2026. *A progressive Node.js framework*. [Online]
Available at: <https://docs.nestjs.com>
[Accessed 3 August 2026].
- Spring, 2026. *Spring boot*. [Online]
Available at: <https://docs.spring.io/spring-boot/index.html>
[Accessed 3 August 2026].
- Tosi, D., 2024. Studying the quality of source code generated by different ai generative engines: An empirical evaluation.. *Future Internet*, 16(6), p. 188.

AI Research & Verification Record

Team member	AI Tool/ Use	Purpose	Output used?	How independently verified	What was changed/ rejected
Gerald	Gemini	To find valid sources for my part	Yes	Input the sources provided into google scholar and make sure that they exist and are actually relevant.	Rejected 5 sources for not being able to find them on google scholar.
Mogau	Perplexity	Find good sources to support or criticise my suggestions.	Yes	I went to the websites of the sources and read through the information to check if I can use it or not.	Out of the 4 sources given only 2 were used, because the rejected one was vague and more business orientated and did not give enough technical information.